UNIVERSIDADE PAULISTA (UNIP)

CST em Análise e Desenvolvimento de Sistemas — PIM II

Relatório Técnico — Etapa 5: Programação Estruturada em C (`sistema_loja.c`)
Projeto: Sistema de E-Commerce CyberHardware
Disciplina: Projeto Integrado Multidisciplinar II (PIM II)
Aluno: Vittor
Campus: Mirassol / SP — 2026

---

1. Introdução e Alinhamento Integral à Etapa 3 (`PIM.pdf`)

A Etapa 5 implementa o núcleo lógico (*Controller* e *Model*) do sistema **CyberHardware E-Commerce** em **Linguagem C estruturada (C99/Win32 Winsock2)** no arquivo `sistema_loja.c`, operando tanto via servidor HTTP/REST local na porta `8080` (com interface SPA em `web/index.html`) quanto via interface de terminal (CLI).

Em conformidade estrita com o documento base de Engenharia de Software (Etapa 3 — `PIM.pdf`), a aplicação implementa todos os Requisitos Funcionais (`RF01` a `RF06`), Requisitos Não Funcionais (`RNF01` a `RNF06`) e Casos de Uso (`UC01` a `UC08`):

Requisito / UC (Etapa 3)	Implementação Estruturada em `sistema_loja.c`	Endpoints / Ações
RF01 / UC01 / UC02 / RNF01	Cadastro de `Usuario`, login com *Bearer Token*, atualização de perfil, inativação lógica e hash irreversível **SHA-256 com salt** (`sha256_hex()`).	`POST /api/auth/register`, `POST /api/auth/login`, `PUT /api/auth/profile`, `POST /api/auth/inactivate`
RF02 / UC03 / UC06	Gestão do catálogo (`Produto`) com preço, preço anterior, especificações, categoria, imagem SVG (`/img/*.svg` com `alt` WCAG 2.1) e ativação/inativação por Admin.	`GET /api/produtos`, `POST /api/produtos`, `PUT /api/produtos`
RF03 / UC07	Monitoramento de `estoque <= estoque_minimo` com alertas em painel e console, além de ajuste manual administrativo.	`terminal_alertas_estoque()`, `PUT /api/produtos`, `GET /api/status`
RF04 / UC04	Estimativa de frete regional por CEP e quantidade de volumes no backend (`calcular_frete_pedido()`), além de regras de desconto (`TECH10`, `CYBER20`).	`POST /api/frete`
RF05 / UC05	Checkout autenticado (`<<include>> UC02`) em 2 fases integrado à simulação de **Gateway de Pagamento** (`PIX`, `CARTAO`, `BOLETO`) e proteção contra estoque negativo.	`POST /api/checkout`
RF06 / UC08 / RNF06	Persistência estruturada em arquivos, geração de código de rastreamento (`CHW...BR`) e histórico com evolução logística (`Pagamento Aprovado`, `Em Separacao`, `Enviado`, `Entregue`).	`GET /api/pedidos`, `PUT /api/pedidos/status`, `pedidos_loja.txt`

2. Estruturas de Dados (`struct`) Modeladas em C

O sistema mapeia diretamente as classes definidas na Etapa 3 para 6 registros compostos (`struct`) em `sistema_loja.c`:

1. **`Usuario`**: representa a generalização de `Cliente` e `Administrador` (`id`, `nome`, `cpf`, `email`, `telefone`, `endereco`, `papel`, `nivel_acesso`, `ativo`, `senha_hash[96]`). As credenciais jamais são armazenadas ou trafegadas em texto claro após o registro.
2. **`Produto`**: armazena `id`, `nome`, `categoria`, `preco`, `preco_antigo`, `estoque`, `estoque_minimo`, `ativo`, `badge`, `foto` e `specs`.
3. **`ItemCarrinho`**: compõe os itens de cada compra (`id_produto`, `nome_produto`, `quantidade`, `preco_unitario`, `subtotal`).
4. **`Pagamento`**: encapsula a resposta do Gateway (`forma_pagamento`, `status_transacao` e `codigo_autorizacao`).
5. **`Pedido`**: agrega `id_pedido`, `id_usuario`, `codigo_pedido`, endereço/CEP, vetor `itens[15]`, `valor_bruto`, `desconto`, `valor_frete`, `valor_total`, subestrutura `Pagamento`,

`status_pedido`, `codigo_rastreamento` e `data_hora`.

6. **`SessaoAtiva`**: controla tokens em memória (`token[65]`, `id_usuario`, `expira_em`) com expiração de 4 horas.

---

3. Segurança Criptográfica em C Puro (`SHA-256`) e Integridade Transacional

Para cumprir o **RNF01** e a **LGPD** sem depender de bibliotecas externas ao compilador MinGW/GCC, implementou-se integralmente o algoritmo **FIPS 180-4 SHA-256** em `sistema_loja.c`:

- Na criação ou atualização de senha (`gerar_hash_senha()`), o sistema gera um *salt* hexadecimal de 16 caracteres e calcula `SHA256("CYBERHW:" + salt + ":" + senha)`, persistindo `<salt>$<digest_64hex>` em `usuarios_loja.txt`.
- Na serialização JSON (`serializar_usuario_json()`), o campo `senha_hash` é deliberadamente omitido para impedir vazamento de resumos criptográficos na rede.
- Na leitura e escrita de arquivos e JSON, utiliza-se `setlocale(LC_CTYPE, "Portuguese")` combinado com `setlocale(LC_NUMERIC, "C")`, evitando que a formatação numérica do Windows substitua pontos decimais por vírgulas em cargas JSON (`%.2f`).

Fluxo Atômico em Duas Fases e Gateway de Pagamento

Conforme o Diagrama de Sequência da Etapa 3 (`PIM.pdf`, p. 11), `executar_checkout_atomico()` assegura que nenhuma baixa de estoque ocorra antes da validação integral de saldos e da aprovação financeira no Gateway:

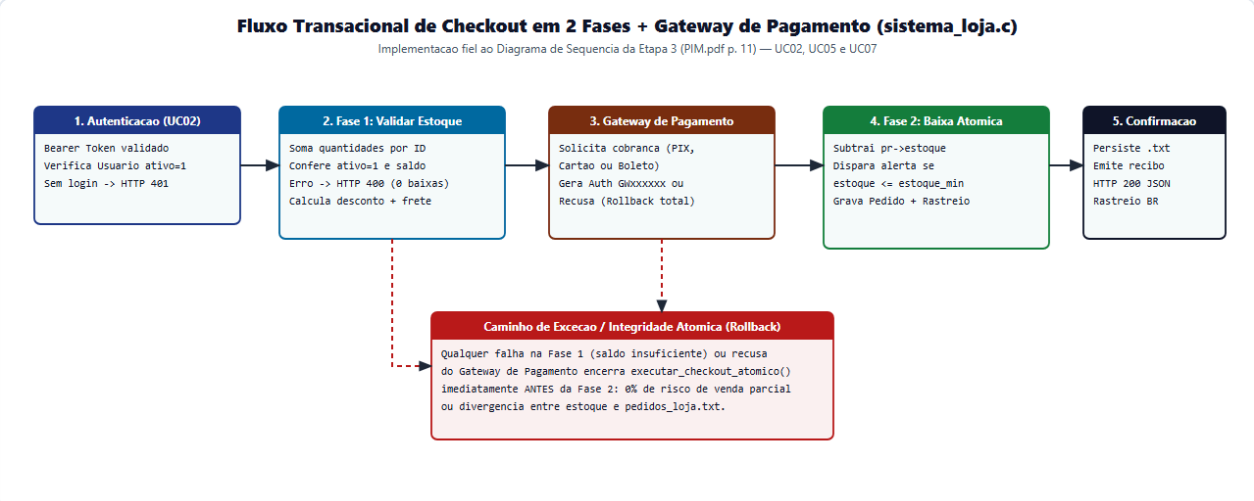


Figura 1 – Fluxo transacional de checkout em duas fases integrado ao Gateway de Pagamento.

---

4. Tabela de Funções Modulares de `sistema_loja.c`

Função	Responsabilidade / Regra de Negócio
`sha256_hex()` / `gerar_hash_senha()` / `verificar_hash_senha()`	Hash irreversível SHA-256 com *salt* e comparação em tempo constante (`RNF01`).
`carregar_*_arquivo()` / `salvar_*_arquivo()`	Leitura e gravação segura com sanitização de delimitadores (`usuarios_loja.txt` , `produtos_estoque.txt` , `pedidos_loja.txt`).
`calcular_frete_pedido()`	Calcula o frete pela primeira região do CEP (`0` .. `9`) e total de unidades, aplicando isenção em compras `>= R$ 8.000,00` (`RF04`).
`processar_gateway_pagamento()`	Valida `PIX` , `CARTAO` ou `BOLETO` , emite código `GWxxxxxx` ou retorna recusa sem alterar estoque (`UC05`).
`executar_checkout_atomico()`	Executa Fase 1 (validação de usuário, CPF, itens repetidos e saldos) -> Gateway -> Fase 2 (baixa atômica, alerta de estoque mínimo `<= estoque_minimo` e geração de rastreamento `CHW...BR`).
`tratar_requisicao_http()`	Faz leitura completa do payload via `recv()` usando `Content-Length` case-insensitive, valida *Bearer Token* e roteia `GET` / `POST` / `PUT` REST.
`servir_svg_produto()`	Gera ilustrações vetoriais (`image/svg+xml`) para `/img/*.svg` , garantindo imagens acessíveis offline com `alt` descritivo (`RNF03`).

5. Compilação, Execução e Evidências de Teste

O código-fonte utiliza apenas bibliotecas padrão da linguagem C e a API nativa `Winsock2` do Windows, sendo compilado com zero avisos pelo GCC (MinGW):

```
gcc -Wall -Wextra -O2 -static sistema_loja.c -o sistema_loja.exe -lws2_32 -lshell32
```

- **Modo Interativo:** executar `EXECUTAR_SISTEMA_C.bat` ou `sistema_loja.exe` exibe o menu com 6 opções operacionais e inicia o servidor na porta `8080` (com fallback automático até `8089` caso a porta `8080` esteja ocupada localmente).
- **Modo Servidor Dedicado:** `sistema_loja.exe --server-only --port 8089` inicia exclusivamente a API REST e a interface SPA.

Evidência Real de Recibo Persistido (`recibos_pedidos.txt`) e Latências (`RNF04`)

Nos 16 testes automatizados de integração ponta a ponta executados contra `sistema_loja.exe` , todas as rotas responderam entre **0,42 ms e 55,12 ms** (muito abaixo do limite máximo de `<= 2 segundos` especificado no `RNF04`), validando recusa de Gateway sem perda de estoque (`HTTP 400`),

checkout aprovado (`HTTP 200`), disparo automático de alerta de estoque mínimo e inativação de conta (`HTTP 403` no login subsequente).

```
============================================================
PEDIDO: CHW-000110 (ID #1) | DATA: 13/11/2026 14:03:30
CLIENTE (UID #3): Mariana Teste PIM | CPF: 98765432100
E-MAIL: mariana@teste.com.br | TEL: (11) 96666-5555
ENTREGA: Av. Brasil, 200 - Rio de Janeiro/RJ (CEP: 22041-001)
PAGAMENTO: PIX | GATEWAY: APROVADO (AUTH GW589413)
STATUS PEDIDO: Pagamento Aprovado | RASTREIO: CHW107919BR
------------------------------------------------------------
 - 2x Memoria RAM Kingston Fury Beast 32GB DDR5 | Unit: R$ 899.90 | Sub: R$ 1799.80
 - 3x SSD NVMe WD Black SN850X 2TB PCIe 4.0 | Unit: R$ 1149.90 | Sub: R$ 3449.70
------------------------------------------------------------
SUBTOTAL: R$ 5249.50 | DESCONTO: -R$ 524.95 | FRETE: R$ 49.90
TOTAL PAGO: R$ 4774.45
============================================================
```

---

6. Referências

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro, 2011.

DEITEL, Paul; DEITEL, Harvey. **C: Como Programar**. 6. ed. São Paulo: Pearson Prentice Hall, 2011.

KERNIGHAN, Brian W.; RITCHIE, Dennis M. **C: A Linguagem de Programação**. Padrão ANSI. Rio de Janeiro: Campus, 1989.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). **FIPS PUB 180-4**: Secure Hash Standard (SHS). Gaithersburg, MD, 2015.